Projects & Experience - Deep Dive

Every project explained end to end, with the technology choices

Nirmalya Mandal - SAP Labs Interview Prep

Study pack - part 2 of 8

Contents

How to use this document

This is the most important part of the pack, because the seniors were unanimous: **most questions come from your projects, and you must be able to defend every line.** For each project you get:

- **In plain words** - what it is, as if explaining to a non-technical friend.
- **How it works** - the flow, step by step.
- **The stack, and why** - every technology, and honestly why you picked it over the obvious alternative. This is where interviews are won, because "why this and not that" shows you think, not just copy.
- **Likely cross-questions** - what they will probably dig into, and how to answer.

Golden rule from the seniors: in the interview, give a short overview and let them ask. Do not dump everything at once. This document is so *you* know everything; you reveal it a layer at a time.

A note on honesty: describe trade-offs as real engineering decisions. "I chose X because of A and B; the cost was C, which was acceptable here because D." That sentence pattern makes you sound like an engineer, not a tutorial-follower.

DokLink - your flagship (talk about this first)

Your role: Chief Technology Officer and sole developer. **Status:** live on the Google Play Store. This is the project to lead with - it is real, it is yours end to end, and it is on an app store.

In plain words

DokLink is an **emergency healthcare app** that helps someone in a medical emergency **find and book a hospital bed in real time**. You open it during an emergency, it shows nearby hospitals sorted by distance with live counts of available general and ICU beds, and you can reserve a bed with a countdown timer sized to how long it will take you to get there. I built all of it - the mobile app, the backend, and the servers it runs on.

How it works (the flow)

1. A user triggers an emergency in the app.
2. The app finds **nearby hospitals sorted by distance**, each showing live **general** and **ICU** bed counts.
3. The user reserves a bed. The system gives them a **reservation window** (a countdown) based on their travel distance - more distance, more time.
4. If they cancel, get admitted, or the timer expires, the bed is **automatically released** back to the pool.
5. Payments, identity verification, and access control wrap around all of this.

The stack, and why

Mobile app: React Native + Expo (with TypeScript)

- **React Native** lets me build a real Android app using JavaScript/TypeScript and React - the same skills I already use for web. One language and one mental model across my whole stack meant I could build fast, alone, under real pressure.
- **Why not Flutter?** Flutter is excellent and arguably gives smoother UI out of the box, but it uses **Dart**, a language I would have had to learn from scratch. React Native let me reuse everything I already knew from React web development, and it has a massive library ecosystem. For a solo developer who needed to ship a working healthcare app quickly, **skill reuse and speed beat Flutter's UI polish**. That was the trade-off, made consciously.
- **Why Expo?** Expo sits on top of React Native and handles the painful parts - building the app, over-the-air updates, and native modules - without me maintaining a lot of native tooling. For a one-person team that is a huge time saver.

Backend: Django + Django REST Framework (DRF), in Python

- **Django** is a "batteries-included" framework - it comes with an admin panel, an ORM (database layer), authentication scaffolding, and security defaults built in. As a solo developer, that meant I did not have to assemble twenty libraries myself.
- **DRF** turns Django into a clean REST API backend for the mobile app to talk to.

- **Why not Node/Express?** Node is great and I use it elsewhere (the glass project), but it is minimal - you build everything yourself. For a healthcare backend with auth, an admin interface, and a lot of business rules, **Django's built-in structure and safety let one person move faster and make fewer security mistakes**. Node would have meant more wiring for the same result.

Database: PostgreSQL

- A **relational** database, because DokLink's data is highly structured and relationship-heavy: users, hospitals, beds, reservations, payments - all linked, all needing consistency.
- **Why not MongoDB?** MongoDB (a NoSQL document database) is flexible and good for loosely-structured data, but the bed-booking core **needs strong guarantees**: a bed count must never go wrong, and a reservation must link reliably to a user and a hospital. PostgreSQL gives me **transactions and relational integrity**, which is exactly what a booking system lives and dies on. Flexibility was not my problem; correctness was.

The hard part - the race condition (this is your best story):

- The booking flow is a textbook **race condition**: two users try to reserve the last ICU bed at the exact same moment. If handled naively, both succeed and the bed count goes negative.
- I solved it with **atomic database transactions** and **row-level locking**, so that only one reservation can win and the bed count can never go below zero. Plus **automatic expiry**: if a reserved bed is not used within its window, a scheduled job returns it to the pool.
- This is the single most impressive technical thing on your resume. Be ready to explain it slowly and clearly (there is a full explanation in the DSA/fundamentals doc under transactions).

Security and identity

- **JWT (JSON Web Token)** for authentication - after login the user gets a signed token they send with each request, so the server does not need to store sessions.
- **OTP (One-Time Password)** verification and **Aadhaar validation** to confirm real identity, which matters for a healthcare app.
- **Role-based access control** so different users (patients, hospital staff, admins) see and do different things.

Payments: Razorpay with HMAC-SHA256 verification

- **Razorpay** is a popular Indian payment gateway - the natural choice for an Indian product.
- After a payment, Razorpay sends back a signature. I verify it using **HMAC-SHA256** - a cryptographic check that proves the payment confirmation genuinely came from Razorpay and was not forged. I manage the full transaction lifecycle (created, paid, verified, failed).

Media and jobs: Cloudinary + scheduled tasks

- **Cloudinary** stores and optimises uploaded images (like documents) instead of putting them on my own server.
- **Scheduled jobs** handle time-based work - most importantly, expiring stale reservations.

Infrastructure: Docker + Nginx on a Linux server

- **Docker** packages the app so it runs the same everywhere.
- **Nginx** sits in front as a **reverse proxy** - it receives web traffic and forwards it to the app, and handles HTTPS.
- **Why a single monolith and not microservices?** For a small team shipping fast, **one well-organised codebase beats a distributed system nobody has time to operate.** Microservices add network calls, deployment complexity, and operational overhead that a solo developer cannot justify. I chose the boring, reliable option on purpose.

Likely cross-questions

- **"Explain the race condition and how you fixed it."** -> Two users, last bed, same instant. Atomic transaction + row lock so only one wins; count never goes negative; unused reservations auto-expire.
- **"Why React Native over Flutter?"** -> Skill reuse from React web, faster solo development, huge ecosystem; Flutter's Dart was a learning cost I did not need. (See above.)
- **"What is a JWT and why use it?"** -> Signed token carrying identity; stateless auth; server verifies the signature instead of storing sessions.
- **"What does HMAC-SHA256 verification protect against?"** -> A forged payment-success message. Only Razorpay and I share the secret, so only a genuine confirmation produces a valid signature.
- **"You're CTO as a student - what does that really mean you did?"** -> Be honest and grounded: sole developer and technical owner of a real startup's product, from design to a live Play Store app and the server it runs on. Ownership, not a title for show.

FloatChat - your AI project

What it is on the resume: an AI conversational analytics platform for querying large-scale Argo oceanographic data in natural language. This is your "I can build generative-AI workflows" proof, and it maps directly onto what SAP does with Joule.

In plain words

FloatChat lets an ocean researcher **ask a question in plain English** - like "show me salinity near the equator in 2015" - and get a real answer from a huge scientific dataset, shown as maps, depth charts, and a 3D globe. Instead of writing database queries by hand, you just talk to it. The data is **Argo** - a global fleet of floating ocean sensors - covering many years of readings.

How it works (the flow)

1. The user types a question in natural language.
2. A **large language model** converts that question into an **SQL database query**, guided by knowledge of the data's structure.
3. The query is **validated before it runs** (so a bad or unsafe query is caught).
4. It runs against a PostgreSQL database and returns results.
5. Results are turned into **visualisations** - 2D maps, depth profiles, and a 3D globe.
6. For follow-up questions, the system uses **RAG** to pull in relevant context so it understands the conversation.

The stack, and why

Frontend: Next.js. Backend: FastAPI (Python).

- **Why a separate FastAPI backend instead of doing it all in Django or Next.js?** The heavy work here is **AI and data processing**, which lives in the Python ecosystem (LangChain, model libraries, scientific data tools). **FastAPI** is a modern, fast, lightweight Python framework built for exactly this - APIs that call models and process data - with automatic validation and great async support.
- **Why FastAPI and not Django here, when I used Django for DokLink?** Different jobs. DokLink needed Django's built-in admin, ORM, and structure for a big business app. FloatChat needed a **lean, fast API layer to orchestrate AI calls** - Django's weight would be dead weight. Picking the right tool per project is itself the point.

The AI: LangChain orchestrating Mistral 7B (via HuggingFace)

- **LangChain** is the framework that wires the steps together - take the question, add schema context, call the model, validate, run, return.
- **Mistral 7B** is an open large language model (7 billion parameters) I ran through HuggingFace.
- **Why Mistral 7B and not a frontier model like GPT-4?** A 7B open model gives **predictable cost and latency** - important when every single chat message triggers an AI call. A frontier model would generate slightly better SQL but at higher, less predictable cost. I made it reliable with **constraint instead of raw power**: schema-aware prompting (the model is told the exact

table structure), query validation before execution, and retrieving similar past queries as examples. The lesson - a smaller model plus good engineering can beat a bigger model used carelessly.

RAG - Retrieval-Augmented Generation, with a vector database (Supabase)

- **RAG** means: before answering, retrieve relevant context and feed it to the model so its answer is grounded in real information, not guessed.
- A **vector database** stores text as numerical "embeddings" so you can find items by **meaning**, not exact words. FloatChat retrieves similar earlier queries to guide the model. **Supabase** provides both the PostgreSQL database and the vector storage.

Data: ETL pipelines for NetCDF

- The raw Argo data comes as **NetCDF** files (a scientific data format). I built **ETL** pipelines - **Extract, Transform, Load** - to read those files, reshape the data, and load it into PostgreSQL.
- **Why pre-process once instead of reading files live?** Because then every user query hits **fast, indexed database tables** instead of slow raw files. Do the expensive work once, up front.

Visualisation: 2D maps (Leaflet) and a 3D globe

- Scientific data is spatial, so it has to be shown on maps and a globe. The heavy shaping of visualisation data is done on the server to keep the browser light.

Likely cross-questions

- **"What is RAG?"** -> Retrieval-Augmented Generation: fetch relevant context first, then let the model answer grounded in it, instead of relying on the model's memory.
- **"Why a small model over GPT-4?"** -> Predictable cost/latency per message; made reliable with schema-aware prompting and validation. Engineering over raw size.
- **"What is a vector database / embedding?"** -> Text turned into numbers that capture meaning, so you can search by similarity of meaning rather than exact keywords.
- **"What is ETL?"** -> Extract data from a source, Transform it into the shape you need, Load it into your database. I used it to move NetCDF files into PostgreSQL.
- **This is your bridge to SAP:** "FloatChat is basically the same pattern as SAP's Joule - natural language in, real queries over real data out. I've built a generative-AI workflow, just on ocean data instead of business data."

A Fashions - production e-commerce (No Strategy Studios)

What it is: a premium e-commerce platform for an international leather-goods manufacturer, built end to end during your time at No Strategy Studios. Your proof of **security hardening, SEO, and running your own server.**

In plain words

A polished online store for a leather brand. I built the whole storefront, made it secure against the abuse any public site faces, got it to a perfect SEO score, and set up and ran the Linux server it lives on.

The stack, and why

Frontend: Next.js + TypeScript + Tailwind CSS

- **Why Next.js instead of plain React?** For a shop, **SEO and load speed are money** - customers have to find you on Google and pages must load fast. Next.js does **server-side rendering** (pages are built on the server so search engines and users get full content immediately), whereas plain React ships a blank page that fills in later, which is worse for SEO and first load. That single difference is why an e-commerce site uses Next.js.
- **TypeScript** adds types to JavaScript, catching whole classes of bugs before the code runs.
- **Tailwind CSS** is a utility-first styling approach that made building a responsive, custom design fast.

Security hardening (this is the differentiator)

- **API rate limiting** in multiple tiers - capping how many requests someone can make, to stop bots and abuse.
- **Captcha** protection on sensitive endpoints, to block automated scripts.
- **XSS (Cross-Site Scripting)** mitigation - stopping attackers from injecting malicious scripts into pages.
- **CSRF (Cross-Site Request Forgery)** mitigation - stopping a malicious site from making requests as a logged-in user.
- Strict **security headers** for production.
- **Say it like this:** "A public shop lives on the open internet with bots and scrapers hitting it constantly. I layered rate limiting, captcha, and strict headers so it survives that."

SEO: a 100 Lighthouse score

- **Lighthouse** is Google's site-quality tool; 100 is a perfect score. I hit it using **structured metadata**, **Schema.org** structured data (which tells search engines exactly what each page is), and **dynamic sitemaps** (an auto-updating map of the site for search engines).

Deployment: Linux VPS + Nginx reverse proxy + automated SSL

- **VPS** = Virtual Private Server - a server I fully control.

- **Why a VPS and not managed hosting (like Vercel)?** Managed hosting is easier but costs more and gives less control. A VPS meant **full control of the stack at a fraction of the cost** - the trade-off being that I take on the operational responsibility, which I was happy to own.
- **Nginx** as a **reverse proxy** receives traffic and forwards it to the app; **automated SSL** (via Certbot) keeps HTTPS certificates valid so the site is always secure.

Likely cross-questions

- **"Why Next.js for e-commerce?"** -> Server-side rendering for SEO and fast first load; plain React can't match that for a public shop.
- **"Difference between XSS and CSRF?"** -> XSS injects malicious scripts into your page (attacking the user through your site); CSRF tricks a logged-in user's browser into making an unwanted request (abusing their existing session). Different attacks, different defences.
- **"What is a reverse proxy?"** -> A server (Nginx) that sits in front of your app, receiving all requests and forwarding them, while handling HTTPS, and load distribution.
- **"What is rate limiting and why?"** -> Capping requests per user/IP over time to stop brute-force and bot abuse.

Glass Workflow Automation - Modern Mahal

What it is: a workflow automation platform that digitises a glass-manufacturing business, from a customer's WhatsApp sketch all the way to production tracking. Your proof of **applying AI to a messy real-world business problem** - very close to what SAP does.

In plain words

A glass factory used to take orders as **rough hand-drawn sketches over WhatsApp**, then work everything out on paper. I built a system that pulls those sketches in automatically, uses AI to turn them into **clean, editable digital production diagrams**, checks the measurements make sense, generates a price quote, and tracks the job through production.

How it works (the flow)

1. A customer sends a rough sketch over **WhatsApp**; the system ingests it automatically.
2. An **AI-assisted module** converts the sketch into a **structured, fully editable production diagram** - extracting dimensions, shapes, holes, and cut markings.
3. **Smart validation** checks the geometry: dimension mismatches, overlapping holes, and whether the piece is actually manufacturable.
4. A human (the admin) reviews and approves - the diagram stays editable, nothing goes to the factory without sign-off.
5. The system generates a **quotation** and tracks the job through **production**, with **role-based access control** for different staff.

The stack, and why

Frontend: React. Backend: Node.js + Express. Database: PostgreSQL. Plus an AI processing module.

- **React** for a rich, interactive diagram editor in the browser.
- **Why Node + Express here, when DokLink used Django?** This app is lighter on built-in business scaffolding and heavier on **custom real-time logic and integrations** (WhatsApp media sync, diagram processing). **Express** is minimal and flexible - I wanted to shape the flow myself rather than fit into Django's conventions. Again: right tool per project.
- **PostgreSQL** because manufacturing data (orders, diagrams, dimensions, jobs) is structured and relational.
- The **AI-assisted module** extracts geometry from messy sketches.

The key engineering judgement - a human in the loop:

- Hand-drawn sketches are **ambiguous by nature**. So the system extracts what it can, **flags what it cannot**, and keeps a human in control: every generated diagram is editable and needs explicit approval before production.
- **Why suggestions and not full automation?** In manufacturing, **a wrong dimension is expensive** - it wastes real glass and money. I deliberately made the AI assist, not decide. That

is mature engineering judgement, and a great thing to say: "I chose to keep a human in the loop because the cost of an automated mistake was too high."

- **Validation runs synchronously on save** so errors appear while the operator still has the context, not after the job reaches the factory floor.

Likely cross-questions

- **"How do you handle a sketch the AI reads wrong?"** -> It flags uncertainty, keeps the diagram editable, and requires human approval. Nothing auto-goes to production.
- **"Why keep a human in the loop instead of full automation?"** -> The cost of a wrong dimension in manufacturing is high; assist beats auto-decide here.
- **"Why Node here and Django for DokLink?"** -> Custom real-time logic and integrations suited Express's flexibility; DokLink's business-heavy backend suited Django's built-in structure.

Vayita Grow - B2B corporate + internal platform (freelance)

What it is: a B2B corporate website plus an internal management platform for an agricultural-inputs manufacturer, built as freelance work, meant to be shown to investors. Your proof of **one codebase serving two very different audiences** - literally digitising how a business runs, which is SAP's whole domain.

In plain words

An agriculture company needed two things: a **professional public website** to show products and let dealers and distributors make inquiries, and an **internal tool** for staff to manage clients, orders, statements, and field reports. I built both in one system, polished enough to show investors that the company runs digitally.

The stack, and why

Next.js (App Router) + TypeScript + Tailwind CSS + Supabase (PostgreSQL).

- **Next.js App Router** so I could build both a public marketing site and a data-heavy internal tool in one project, using **server components** by default (rendered on the server, fast and SEO-friendly) and only making the interactive leaf parts run in the browser.
- **Supabase** gave me a hosted PostgreSQL database with authentication built in - fast to set up for a freelance timeline.
- **Zod-validated server actions** - **Zod** checks that incoming data is exactly the right shape before it touches the database, which keeps the internal tool safe.

The interesting challenge:

- **One codebase, two audiences** - a polished public site for partners and investors, and a dense internal tool for staff. I kept them coherent with clear **route groups** and a shared design system.
- **A conscious trade-off:** I shipped the **highest-value internal modules first** instead of building the entire ERP surface. "An investor demo that works beats a feature list that doesn't." Internal tables **paginate on the server** so they stay fast as data grows.

Likely cross-questions

- **"What are server components?"** -> Components rendered on the server, sending finished HTML - faster and better for SEO; only interactive bits run in the browser.
- **"How did you keep the public site and internal tool separate but consistent?"** -> Route groups for separation, a shared design system for consistency.
- **This is a natural SAP bridge:** "This project was literally about digitising how a business runs its clients, orders, and reports - which is exactly the kind of problem SAP solves, just at a much larger scale."

AUKTAVE 2K26 - event portal

What it is: a production event-management portal for a university tech fest - registrations, schedules, sponsor showcases, event discovery. Your proof of **clean architecture and front-end performance.**

In plain words

The official website for a college fest. People discover events, register, see schedules, and view sponsors. I built it so that **adding a new event is just a data change, not a code change** - the content is cleanly separated from the design.

The stack, and why

Next.js 16 + React 19 + TypeScript + Tailwind CSS + Framer Motion + GSAP + PWA.

- **A typed content architecture:** all event data lives in structured TypeScript files, separate from the presentation, with **nested dynamic routing** so each event gets its own page automatically.
- **Why typed static content and not a CMS (content management system)?** A fest has a **fixed content window** - the events are known ahead of time. Type-checked data files are simpler, faster, and safer than running a whole CMS. Right-sizing the solution.
- **SEO:** structured metadata, Open Graph tags (nice link previews when shared), and generated sitemaps.
- **PWA (Progressive Web App):** the site can be installed and works offline - useful for attendees with patchy phone data.
- **Framer Motion + GSAP** for animation. The engineering judgement: **animation must not wreck performance on mid-range phones**, which is what most attendees carry. So motion is scoped to visible sections, respects "reduced motion" preferences, and is lazy/interruptible rather than blocking the first paint.

Likely cross-questions

- **"What is a PWA?"** -> A web app that can be installed and work offline, using service workers to cache content.
- **"Why not a CMS?"** -> Fixed, known content; typed data files are simpler and safer than CMS overhead.
- **"How do you keep heavy animations fast?"** -> Scope motion to visible sections, respect reduced-motion, keep it lazy and non-blocking.

CONTEXT Language Interpreter - your CS-fundamentals project

What it is: a custom programming-language interpreter built in Python - lexical analysis, parsing, AST generation, and runtime execution. Your proof that you understand **how languages and computation actually work underneath** - a strong signal of fundamentals.

In plain words

I built my own small programming language. You write code in it, and my interpreter reads that code, understands it, and runs it - handling arithmetic, if/else logic, and variables. It is the same pipeline that real languages like Python use, built from scratch so I understood it deeply.

How it works (the four stages - know these)

1. **Lexical analysis (the lexer/tokeniser):** reads the raw text and breaks it into **tokens** - the smallest meaningful pieces, like numbers, operators, and keywords. (`3 + 4` becomes the tokens `3`, `+`, `4`.)
2. **Parsing:** takes the tokens and builds an **AST (Abstract Syntax Tree)** - a tree that represents the structure and meaning of the code, respecting precedence (so `3 + 4 * 2` groups the multiplication first).
3. **AST generation:** the tree is the program's structure in a form the computer can walk.
4. **Runtime execution (the interpreter/evaluator):** walks the tree and actually computes the result, keeping track of variables and evaluating conditions.

- It supports **arithmetic evaluation, conditional logic, recursive parsing, and variable bindings**, using a custom execution engine.

Why this matters and likely cross-questions

- This project shows you understand **compilers/interpreters, trees, recursion, and how code becomes execution** - genuinely impressive fundamentals for a student.
- **"What is an AST?"** -> Abstract Syntax Tree: a tree representation of code's structure that captures meaning and precedence, which the interpreter then walks to execute.
- **"Difference between a compiler and an interpreter?"** -> A compiler translates the whole program into machine code ahead of time; an interpreter reads and executes it directly, step by step. Mine is an interpreter.
- **"What is a token?"** -> The smallest meaningful unit of code the lexer produces from raw text.

Research & Publications - explained simply

You have two co-authored papers on your resume. You may be asked "what was your research about?" or "what was your contribution?" Here is the plain-language version so you can speak to it confidently. Both papers are about **how pedestrians interact with self-driving cars**, studied using virtual reality.

The core idea (both papers)

Self-driving cars (**Autonomous Vehicles, AVs**) have a problem: a normal car communicates with pedestrians through a human driver - eye contact, a wave, a nod. A self-driving car has no driver, so **how does a pedestrian know it's safe to cross?** The proposed answer is an **eHMI (external Human-Machine Interface)** - lights, displays, or signals on the outside of the car that tell pedestrians its intentions.

Studying this in the real world is dangerous and expensive (you can't run a self-driving car at real pedestrians to test it), so the research uses **Virtual Reality / Extended Reality** to simulate it safely.

Paper 1 - the XR book chapter (IGI Global, 2026)

"Enhancing Human-Automated Vehicle Interaction in Complex Environments Using Virtual and Extended Reality Technologies." Published as a chapter in *Practical Applications of Smart Human-Computer Interaction*, IGI Global.

- **What it's about:** the challenges of pedestrian-AV interaction at busy, unpredictable urban intersections, and how **Extended Reality (XR)** simulations let researchers test AV behaviour **safely and cheaply** instead of on real roads.
- It surveys the surrounding ideas: how AVs make decisions, how to model and predict pedestrian behaviour, trajectory prediction, and sensor fusion.

Paper 2 - the VR study (accepted for ICDSNE 2026)

"Evaluating Pedestrian Trust and Decision-Making Across eHMI Modalities in Indian Traffic Using Virtual Reality."

- **What it's about:** an actual experiment. In a VR simulation of **Indian traffic**, participants faced self-driving cars using four different eHMI styles - **Overhead, Projection, Windshield, and Headlight** signals - to see which one pedestrians **trusted** most and crossed most confidently.
- **How it was measured:** with **30+ participants**, using **NASA-TLX** (a standard measure of mental workload), **SUS** (System Usability Scale, a standard usability score), and behavioural data (like how quickly people decided to cross).
- **The finding:** pedestrians who **trusted** the car's signal made **faster crossing decisions** - trust and clear communication genuinely help.

How to talk about it

Keep it humble and clear: "I'm a co-author on two papers about how pedestrians interact with self-driving cars. Since a self-driving car has no driver to make eye contact, we studied external signals on the car - lights and displays - that tell pedestrians it's safe to cross, and we tested them in virtual reality because doing it on real roads would be dangerous. One paper is a broader XR study, and the other is a hands-on VR experiment with over 30 participants measuring which signals people trusted most." If pressed on your specific contribution, be honest about what you actually did (data collection, the VR simulation, analysis, writing) - never overstate it.

The one-minute map of all your work

If you want a single mental picture to walk in with:

- **DokLink** - I build and run real production systems end to end (mobile + backend + servers), including the hard parts like race conditions and payments. Live on the Play Store.
- **FloatChat** - I build generative-AI workflows (natural language to real data), the same pattern as SAP's Joule.
- **A Fashions** - I secure and deploy public production sites and I run my own Linux servers.
- **Glass Automation** - I apply AI to messy real business problems with good engineering judgement (human in the loop).
- **Vayita Grow** - I digitise how a business runs - SAP's exact domain, at small scale.
- **AUKTAVE** - I care about clean architecture and real-world performance.
- **CONTEXT** - I understand fundamentals deeply enough to build a language interpreter.
- **Research** - I can work in a structured team over months and communicate technical ideas.

Every one of these points back to your theme: **a builder who ships real things and is hungry to learn enterprise engineering.**